KKBOX's Music Recommendation Challenge

Abdullah Naeem Jennifer Hung Niroshan Benjamin

Simon Fraser University
Instructor: Nicholas Vincent
April 9th, 2025

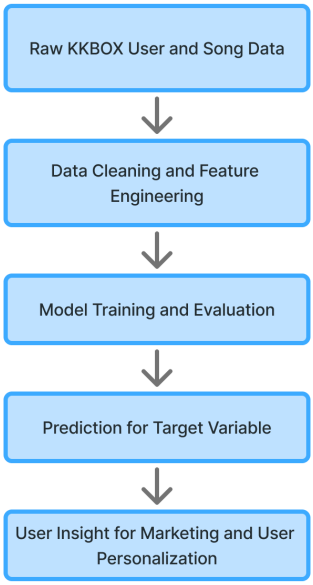
Abstract

Recommendation systems are used worldwide to provide suggestions to users based on their preferences, interests, or past behaviours. Many popular music streaming platforms, such as Spotify and Apple Music, leverage these systems to make the listening experience unique to the user by personalizing content. As avid music lovers, we were motivated to explore how user data might be used to generate customized music recommendations after using Spotify's personalized suggested playlists. This report will provide an overview of our solution to the KKBOX Music Recommendation challenge from the WSDM cup in 2018. Through a process of data cleaning, creating models, and gathering insightful information, our project seeks to assist KKBOX in predicting whether a user will listen to the same song again after a month. Furthermore, this project was based on our interest in working with real data and developing valuable insights that KKBOX can leverage to adapt their marketing strategy or enhance brand awareness.

Keywords

Influence estimation, impact factors, ETL, data processing, LOO influence

Visual Abstract



1 Introduction

In the KKBOX Music Recommendation Challenge for the WSDM 2018 cup [1], participants are asked to develop business insights for KKBOX by designing a recommendation system that predicts

whether a user will revisit a song after a first listening event within a specific time window. A variable ‘target’ is present in the *train.csv* training dataset that has a value of ‘1’ if there was a recurring listening event after one month and a value of ‘0’ otherwise. Other datasets are also included that provide the participant with additional information about users of the service and songs they listen to that might help with our predictions. Our approach to this problem is adapted from the 5-step data processing pipeline, which we followed from the Spending Data Handbook [2]. We understood the problem from the competition, cleaned the dataset provided by KKBOX, created predictive models and evaluated their performance, and finally discussed our results. By following these steps, we hope that our analysis provides valuable insights for KKBOX, helping them target specific users based on their interests.

We will be referencing the formulation for our report and its structure from a similar analysis done by Zhang and Fogelman-Soulié (2018) [3], two former participants of the challenge. Our goal is to present our research and findings to entry-level data analysts and anyone who is interested in learning about how to derive actionable business value from raw data.

2 Related Work

Previous music recommendation analyses. Zhang and Fogelman - Soulié (2018) [3] provide a comprehensive analysis of their solution to the KKBOX Music Recommendation challenge in which they emphasize the importance of feature engineering in data analysis work. Their findings inspired us to structure our report in a similar way by providing a detailed description of all of our modelling choices and performance measures. We will also be relating our work to findings from Zhu et. al (2018) [5], a group of members also involved in the WSDM 2018 challenge. We will also be referencing the work done by these authors, as their methods and abstract relate closely to our goal.

Influence Estimation. We will be conducting influence estimation on our findings, which we adapted from Hammoudeh, Z. and Lowd, D. (2024) [4]. In this survey, the authors describe several different influence estimation techniques that we will be using to assess the influence scores for our chosen predictive models. In particular, our focus will be on LOO estimation and group-based influence.

3 Methods

3.1 Dataset description

The dataset for this study was obtained from the KKBOX Music Recommendation Challenge hosted on Kaggle. KKBOX is a prominent music streaming service in Asia. The dataset contains user-music interaction logs, including membership details, song information, and listening behaviour. The primary objective for participants given the dataset is to predict whether a user will replay a song within 30 days of an initial listening event - making the prediction task a binary classification problem.

The dataset consists of five .csv files. The main file, *train.csv*, shows which songs users listened to and whether they replayed them within 30 days (our target prediction). Rather than using the provided test dataset, we implemented an 80-20 split on the training data to create our own training and validation sets to ensure proper evaluation while maintaining the original data distribution.

There were also additional files provided:

1. Members.csv - User demographic (age, gender, city) and registration information
2. Songs.csv - Comprehensive song metadata, including duration and genre
3. Song_extra_info.csv - Add-on to songs.csv that includes each song's *ISRC* (International Standard Recording Code)

3.2 Data Preprocessing

We performed comprehensive data cleaning to prepare the raw dataset for analysis. First, we loaded the data using memory-efficient data types, converting categorical variables like user IDs and song IDs to the 'category' dtype to reduce memory usage. For the training data, we analyzed missing values, dropping rows with less than 1% missingness in critical columns like 'source_system_tab' while retaining other records by imputing missing values. We removed 4.2% of duplicate user-song interactions to prevent data leakage.

For user data, we first normalized age values by restricting unrealistic entries below 5 and above 105 years, then calculated membership durations from the registration and expiration dates. The song metadata underwent careful processing where we extracted primary genres from complex strings and standardized artist/composer information while also normalizing song lengths and removing outliers.

Through systematic left joins, we meticulously merged the user demographics, song characteristics and listening interaction records, preserving all original training examples while maintaining data integrity. Missing values were handled through a tiered approach: textual fields received "Unknown" placeholders, numerical variables used median imputation, and categorical features adopted mode replacement. Each transformation was carefully documented and validated to ensure consistency.

After completing all cleaning and transformation steps, we saved the processes dataset in a csv file. This systematic preprocessing successfully transformed the raw data into an analysis-ready format optimized for machine learning modeling.

3.3 Model Training

Before we could train different models and assess their performance, we had to decide how to overcome an issue we were having with the scale of the merged dataset. In particular, we had to sample 1000 rows from the merged dataset before we could perform

an 80/20 split. We then split our dataset into a training and validation set using model selection functionality from Python's scikit-learn library.

We decided to compare the performance of 4 predictive models:

1. Gaussian Naive Bayes
2. Logistic Regression
3. Random Forests
4. LightGBM

These models were chosen for our analysis based on a number of factors, including ease of interpretability, robustness to overfitting, training speed, and how well the model fits with the structure of our dataset. Gaussian Naive Bayes was chosen as a baseline, given its training speed and effectiveness in managing high-dimensional data from one-hot encoding. Additionally, the model is a good starting point for classification tasks where the target variable is binary (whether a user will replay a song). Our second model, Logistic Regression, was chosen since the probability outputs of this model integrate well with one of our evaluation metrics: AUC-ROC score. This enables us to evaluate both the model's prediction accuracy and its level of confidence in its predictions. Random Forests were chosen since it captures non-linear relationships present in the dataset and because it can handle mixed data types well. Furthermore, it is resistant to overfitting and generally has more predictive power compared to our other two models. Lastly, we selected the LightGBM model because of its effectiveness, high classification task performance, and capacity to manage both numerical and categorical features. Key parameters were adjusted to balance generalization, model complexity, and learning speed.

We tuned hyperparameters for Logistic Regression by specifying the maximum number of iterations and by adding a random state for the reproducibility of results. For Random Forests, we tuned for the number of trees, the maximum depth of each tree, and the minimum number of samples when making a split to avoid overfitting.

We fitted each model on the training data and computed the accuracy scores on the validation data. We also compared ROC scores for each model as a way to provide an alternative performance measure. Once we were able to determine the best-performing model, we created a confusion matrix that showed the number of false positives, false negatives, true positives, and true negatives. This was a useful data visualization that gave us insights into the classification accuracy of our optimal model.

3.4 Influence Analysis

3.4.1 Leave-One-Out (LOO) Influence

To identify highly influential individual observations in our dataset, we performed a Leave-One-Out (LOO) analysis by systematically excluding each training instance and measuring the change in model performance.

To compute the LOO influence scores on the AUROC score, we will first randomly select 10 different observations from the training dataset. For each selected point, we removed it from the training set, retrained the Naïve Bayes, Logistic Regression, and Random Forest models, and then calculated the new AUROC score. The influence score for each point was determined by measuring the difference between the baseline AUROC score (obtained using the full dataset) and the new AUROC score after removal.

3.4.2 Group-level influence

The group influence analysis examined how removing progressively larger subsets of training data affected model performance. Unlike the leave-one-out analysis that evaluates individual observations, this approach assesses the collective influence of user groups by systematically ablating random subsets from 5% to 55% of the training data in 5% increments. For each size, we retrained all three model architectures - Naïve Bayes, Logistic Regression, and Random Forest - on the reduced datasets and measured the absolute change in AUROC performance compared to the baseline models trained on complete data.

4 Results (TODO)

4.1 Performance Comparison of ML Methods

The following table lists the accuracy and ROC scores both the training and testing datasets for each of our 4 model choices:

Method	Training Accuracy	Testing Accuracy	ROC Score
Naive Bayes	0.52875	0.505	0.529
Logistic Regression	0.535	0.49	0.512
Random Forest	0.864	0.625	0.674
Light GBM	0.765	0.665	0.677

Table 1: Performance Comparison of ML Methods

The Naive Bayes model achieved a 52.9% training accuracy, 50.5% test accuracy, and a ROC-AUC score of 0.529. From these values, the model's performance is slightly better than random guessing. Since user characteristics and song attributes are likely correlated in our dataset, it does not support the fundamental premise of Naive Bayes, which is that features are independent. As a result, the underlying relationship between user behaviour and the likelihood of repeating a song is difficult for the model to capture.

The Logistic Regression model achieved a 53.5% training accuracy, 49% testing accuracy, and an ROC-AUC score of 0.5117. Since the testing accuracy is lower than 50%, the model's performance is based on random guessing. The complexity and non-linear relationships in the dataset likely contributed to the model's difficulties in achieving high predictive performance.

The Random Forest classifier achieved an 86.4% training accuracy, 62.5% testing accuracy, and an ROC-AUC score of 0.674. Overall, this classifier performed noticeably better than the other models. These findings show that the model is able to find subtle patterns in the data and is capable of differentiating between users who will likely listen to a song again and those who will not. Although the large training accuracy suggests overfitting, the test accuracy is still considerably better than our other models, which highlights the model's robustness and predictive power.

Lastly, the LightGBM classifier achieved a 76.5% training accuracy, 66.5% testing accuracy, and an ROC-AUC score of 0.677. The LightGBM model demonstrated that it was able to extract significant patterns from the data without overfitting. Additionally, the model demonstrates a moderate level of generalization to unknown data with its moderately high testing accuracy. The model's ROC-AUC score of 0.677 indicates that it does a respectable job of differentiating between the two classes, making it useful for classification. Overall, the model is balanced and performed well with our dataset.

4.2 Random Forest Confusion Matrix

A confusion matrix was created for one of our best-performing models.

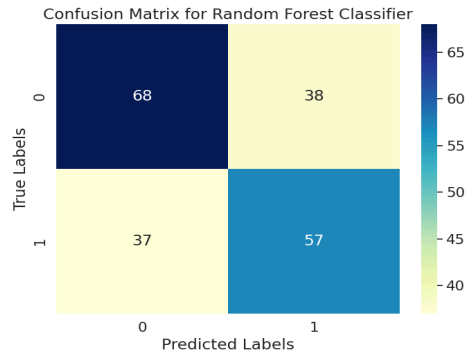


Figure 1: Random Forest Confusion Matrix

The following table displays the number of true positives, true negatives, false positives, and false negatives for the predictions made by the Random forest model:

True Positives	True Negatives	False Positives	False Negatives
57	68	38	37

Table 2: Different Types of Predictions for Random Forest

After a month, the model correctly identified 68 songs as not being replayed and 57 songs as being replayed, according to the confusion matrix. However, it failed to recognize 37 songs that were actually replayed and misclassified 38 songs as replayed when they weren't. Observe that the number of false positives and false negatives are similar; this suggests that the Random Forest model is balanced in its misclassifications and does not suggest that the model is affected by biases towards one prediction class or another.

4.3 Results from Influence Analysis

4.3.1 Influence Scores (LOO Influence)

The brute-force LOO influence analysis revealed significant variation in model sensitivity to individual training observations. As evidenced by the plotted results in Figure 2, Random Forest (RF) exhibited the most pronounced sensitivity, with influence scores reaching AUROC=0.04 for certain critical points, particularly around index 500. This substantial impact suggests these observations may represent either high-value power users whose behavior patterns are particularly informative to the ensemble model, or potential data anomalies requiring validation. The clustered patterns of influence between indices 200-400 further indicate systematic dependencies on specific user subgroups, potentially reflecting underlying biases in data collection or particularly distinctive behavioral cohorts.

In contrast, Logistic Regression (LR) demonstrated more moderate sensitivity, with maximum influence scores not exceeding AUROC=0.01. This relative stability aligns with expectations for parametric models, reflecting their smoother decision boundaries and reduced susceptibility to overfitting individual observations. The most robust performance came from Naïve Bayes (NB), which maintained near-zero influence scores across all points, consistent with its strong independence assumptions and global generalization behavior.This suggests that if we remove a single data point, it has minimal impact on the model's AUROC score. Additionally, Table A in the Appendix shows the computed influence scores for the 10 randomly selected observations.

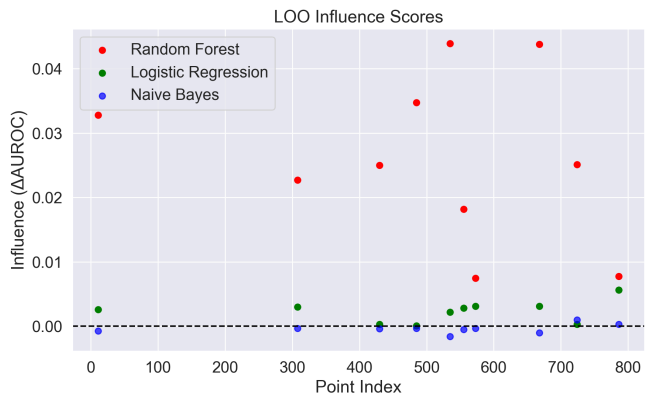


Figure 2: LOO Influences Scores for ML Models

4.3.2 Influence Scores (Group-Level Influence)

The systematic evaluation of group-level influence revealed critical insights into model robustness and data dependence. As shown in Figure 3, all three models exhibited distinct sensitivity patterns when progressively larger subsets of training data (5–55%) were removed. Random Forest demonstrated the strongest performance degradation, with AUROC scores declining nonlinearly as group size increased. This model's AUROC reached 0.04 at just 20% data removal and accelerated sharply thereafter, indicating pronounced vulnerability to data reduction. The ensemble structure of Random Forest, while delivering superior baseline performance, appears heavily reliant on specific data subgroups, making it susceptible to performance drops when critical user segments are excluded.

Logistic Regression displayed more moderate sensitivity, with the AUROC values increasing linearly at approximately 0.002 per 5% removal. This suggests stable generalization capabilities up to 35% data removal, beyond which performance declines became more pronounced (ΔAUROC = 0.02 at 50% removal). The parametric nature of Logistic Regression provides inherent regularization, explaining its balanced trade-off between data efficiency and predictive power.

Notably, Naïve Bayes maintained near-zero AUROC across all group sizes (max ΔAUROC = 0.01 at 55% removal), confirming its robustness to data perturbations. However, this stability likely stems from strong independence assumptions that limit the model's ability to capture feature interactions, as reflected in its lower baseline AUROC (0.742 vs. 0.872 for Random Forest).

Figure 3: Group-Level Influence of ML Models

4.3.3 Data Visualizations Using Tableau

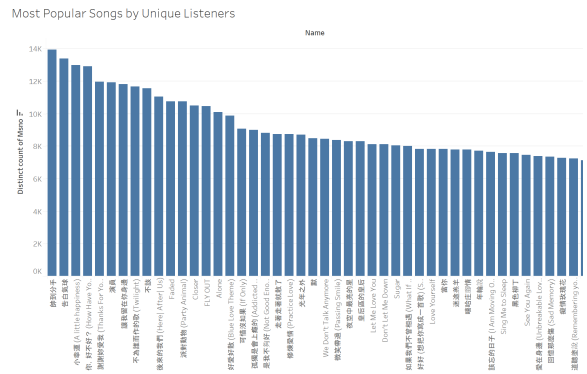


Figure 4: Most Popular Songs by Listener

The above bar plot in Tableau shows what the most popular songs are based on the number of unique music listeners.

Additionally, the bar chart below shows which source recurring listeners are coming from and which ones are most popular based on the number of recurring listeners.

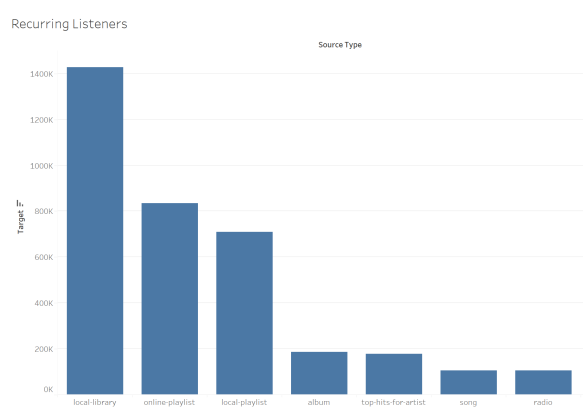
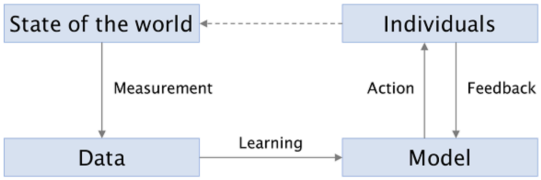


Figure 5: Source Type of Recurring Listeners

5 Discussion

Hammoudeh, Z. and Lowd, D. (2024) [4] provide a comprehensive overview of various influence analysis techniques that can help evaluate whether an observation in a training dataset is influential to a model's predictions. This can help KKBOX determine if they should spend more resources in training models and obtaining accurate results by conducting a more thorough investigation of influential observations. This can help to derive more useful insights and arrive at valuable recommendations when building music recommendation systems tailored to a company's audience.

The FairML Intro [6] discusses the "Machine Learning Loop"



which is a similar view on a pipeline for Machine Learning that aims to minimize demographic disparities. This process follows information from the state of the world, to generating and acquiring data through measurements, developing a model to build predictions, and finally providing actionable insights for individuals who might give feedback to improve model generalizability. Since the predictions gathered from our dataset are based on user demographic data (a relevant feature), it is important that we do not introduce biases in our results by incorrectly weighting observations more for a certain gender or group of listeners compared to another. In order to avoid this, the machine learning loop provides emphasis on feedback loops from individuals to improve model predictions and iterate on results.

To align our findings with data-centric AI principles, our application of influence analysis helped us develop insightful information about the features that influenced our model's predictions. We were able to critically evaluate our data by looking at the features that had the largest effects on the likelihood of a user playing a song again after one month. Influence analysis helps to improve the interpretability of modelling results from a human-centric standpoint, which will help business stakeholders at KKBOX comprehend our systems recommendations more clearly. Furthermore, as we were able to track and eliminate potential biases—like those related to age, location, or user listening habits in our analysis—gathering results on feature influence can help guarantee more equitable recommendations. This will ultimately result in a more moral and accountable recommendation system that we build. Overall, in our project, influence analysis functioned as a link between data-centric enhancements and human-centric accountability, in addition to being a technical assessment tool critical for our results.

In class, we learned extensively about different influence analysis techniques and how the removal of observations from the

training dataset impacts model predictions. Namely we covered each method's mathematical background, use cases, and advantages and disadvantages. What we reviewed in class about influence estimation helped us derive a key portion of our project's contribution and what we did.

6 Overall Artifact Quality (Contributions)

The GitHub link to our repository is: https://github.com/Hav0/CMPT419-Project-.

We broke down our analysis into several key files that allowed us to arrive at valuable insights and conclusions for KKBOX:

1.) *kkbox_data_preprocessing.py* → For data cleaning (encoding variables correctly, removing outliers)

2.) *data_merging.py* → To efficiently merge all the datasets we were interested in to perform a more coherent analysis and fill missing values.

3.) *data_modelling.py* → For creating the models, evaluating the performance, and creating visuals.

4.) *LOO_influence.py* → Performs leave-one-out influence on our training data set

5.) *group_influence .py* → Performs group influence on our training dataset.

6.1 Group Member Contributions

Abdullah: I contributed by helping with the data cleaning to impute missing values and performing data modelling and evaluating performance. I also helped find relevant resources that we can use based on previous competitions and writing the report, where I focused on the abstract, introduction, related work, methods, discussion, and connection to class themes components.

Jennifer: I contributed to preprocessing the dataset by handling missing values and cleaning outliers. I coded the influence scores for LOO influence and Group-Level influence. I also contributed to the report, where I helped structure and articulate the methodology and results.

Niroshan: I found the real donated KKBOX dataset from the competition. I contributed to cleaning the data by filling in the missing values in data_merging.py. I performed data modelling and evaluated performance in data_modelling.py. I also contributed to writing the report by editing and writing various sections such as abstract, visual abstract, introduction, methods, and results.

Note on The Use of AI In This Project

We decided to use ChatGPT and GROK for each component of our analysis. This is because we usually use ChatGPT to help fix our code in general use cases. We also used ChatGPT to help debug errors in the code.

Appendix

Table A - Influence Scores for LOO

Observation	Influence on NB	Influence on LR	Influence on RF
573	-0.000301	0.003111	0.010939
11	-0.000703	0.002609	-0.002559
485	-0.000301	0.000100	0.034123
668	-0.001004	0.003111	0.012395
535	-0.001606	0.002208	0.014151
308	-0.000301	0.003011	0.016710
786	0.000301	0.005620	0.005169
555	-0.000502	0.002810	0.025994
430	-0.000401	0.000301	0.029506
724	0.001004	0.000301	0.015004

References

[1] WSDM Cup. (2018). *KKBOX's Music Recommendation Challenge*. Kaggle. https://www.kaggle.com/c/kkbox-music-recommendation-challenge/overview

[2] Spending Data Handbook (2012). BookType. http://okfn.booktype.pro/spending-data-handbook/_draft/_v/2.0/introduction/

[3] Zhang, J. & Fogelman-Soulié, F. (2018). KKBOX's Music Recommendation Challenge Solution with Feature Engineering. https://wsdm-cup-2018.kkbox.events/pdf/WSDM_KKboxs_Music_Recommendation_Challenge_6th_Solution.pdf

[4] Hammoudeh, Z. and Lowd, D., (2024). Training data influence analysis and estimation: A survey. Machine Learning, 113(5), pp.2351-2403. https://arxiv.org/abs/2212.04612

[5] Zhu et. al (2018) . A Hybrid Approach for Music Recommendation. https://wsdm-cup-2018.kkbox.events/pdf/6_A_Hybrid_Approach_for_Music_Recommendation.pdf

[6] Fair ML Intro. https://fairmlbook.org/introduction.html